



**Politechnika
Śląska**

Helm i Kubernetes

Wprowadzenie Laboratoryjne

Autor

Wojciech Krzyżanowski

Testowanie i weryfikacja

Michał Lach

Rok akademicki

25/26

Wprowadzenie

1. Cel laboratorium i efekty kształcenia

Celem laboratorium jest przeprowadzenie pełnego cyklu: uruchomienia aplikacji Flask w kontenerze Docker lokalnie, następnie wdrożenia tej samej aplikacji do lokalnego klastra Kubernetes przy użyciu Helm, oraz zrozumienie przepływu konfiguracji (ConfigMap) i sekretów (Secret) do aplikacji.

Po ukończeniu laboratorium student/ka potrafi:

1. zbudować obraz Dockera z aplikacji i uruchomić go lokalnie,
2. wdrożyć aplikację do Kubernetes poprzez Helm (chart, values, templating),
3. zdiagnozować podstawowe zasoby klastra (Pods, Deployment, ConfigMap, Secret, Ingress),
4. skonfigurować parametry wdrożenia (replicas, resources, strategy, scheduling, ingress, TLS),
5. uruchomić automatyczną ocenę (grader) i interpretować wyniki.

2. Wymagania wstępne

2.1 Sprzęt i system

Minimalne wymagania (rekomendowane):

1. System: Windows 10/11, macOS lub Linux (x86_64 lub ARM64).
2. RAM: 8 GB (zalecane 16 GB).
3. Dysk: 10 GB wolnego miejsca (obrazy Dockera + dane klastra).
4. Dostęp do internetu (pobranie narzędzi i zależności).

2.2 Oprogramowanie

Wymagane narzędzia:

1. Docker (zalecany Docker Desktop z włączonym Kubernetes).
2. kubectl (CLI do Kubernetes).
3. Helm (menedżer pakietów/charts dla Kubernetes).
4. GNU Make (uruchamianie celów Makefile).
5. Git (pobranie repozytorium).
6. Przeglądarka WWW (weryfikacja działania aplikacji).

3. Instalacja i konfiguracja środowiska

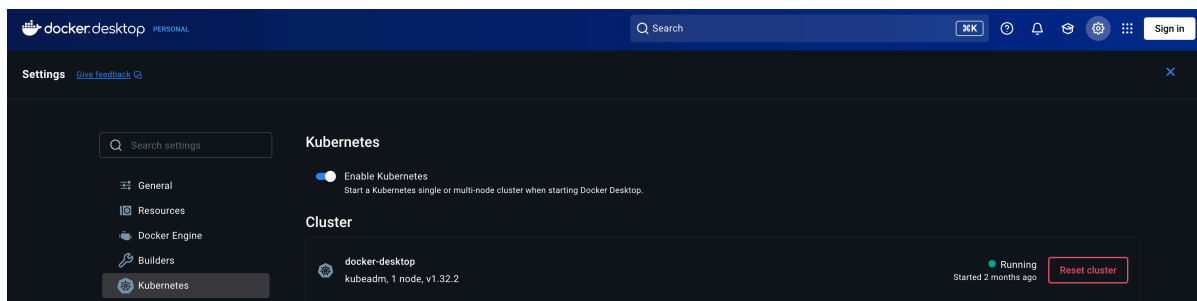
Poniżej podano przykładowe ścieżki instalacji. W środowiskach uczelnianych część narzędzi może być już zainstalowana. W przypadku innej dystrybucji/menedżera pakietów, zastosuj równoważne polecenia.

3.1 Docker Desktop i lokalny Kubernetes

Najprostsza konfiguracja dla tego laboratorium to Docker Desktop z włączonym Kubernetes. W materiałach jako kontekst klastra używany jest: docker-desktop.

Kroki (Docker Desktop):

1. Zainstaluj Docker Desktop dla swojego systemu.
<https://docs.docker.com/desktop/setup/install/mac-install/>
<https://docs.docker.com/desktop/setup/install/windows-install/>
<https://docs.docker.com/desktop/setup/install/linux/>
2. Uruchom Docker Desktop i przejdź do Settings/Preferences.
3. Włącz opcję Kubernetes (Enable Kubernetes) i poczekaj na inicjalizację klastra (nie jest konieczne resetowanie klastra)



4. Zweryfikuj, że działa kontekst kubectl o nazwie docker-desktop.

```
docker version
kubectl version --client
kubectl config get-contexts
kubectl config current-context
```

3.2 Instalacja kubectl i Helm

Jeżeli Docker Desktop nie instaluje kubectl/Helm automatycznie, doinstaluj je osobno.

macOS (Homebrew):

```
brew install kubectl helm
```

Windows (Chocolatey):

```
choco install kubernetes-cli kubernetes-helm
```

Ubuntu/Debian (APT lub w zależności od wersji systemu mogą być wymagane repozytoria producentów):

```
sudo apt-get update
sudo apt-get install -y kubectl helm
```

3.3 Make i pozostałe narzędzia

Make (GNU Make) jest używany jako warstwa orkiestracji poleceń w laboratorium. Na Windowsie możliwa jest konieczność doinstalowania make.

Windows (Chocolatey):

```
choco install make
make -v
```

3.3 Instalacja GitBash

Na systemie operacyjnym Windows może istnieć konieczność doinstalowania GitBash. W tym celu należy skorzystać z oficjalnej instalacji dostępnej przez poniższy link.

<https://git-scm.com/install/windows>

4. Wprowadzenie teoretyczne - Docker

W tej części laboratorium analizujemy **rzeczywisty Dockerfile używany w projekcie**, aby zrozumieć, **jak Docker działa pod spodem** oraz jakie decyzje projektowe wpływają na bezpieczeństwo, wydajność i przenośność aplikacji.

4.0 Dockerfile wykorzystywany w tym projekcie (overview)

```
FROM python:3.13-slim AS builder
WORKDIR /app
RUN python -m venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"
COPY requirements.txt .
RUN pip install --upgrade pip \
  && pip install --no-cache-dir -r requirements.txt --timeout=100 --
  index-url=https://pypi.python.org/simple
FROM python:3.13-slim
WORKDIR /app
RUN apt-get update \
  && rm -rf /var/lib/apt/lists/*
```

```

COPY --from=builder /opt/venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"

RUN addgroup -gid 1000 appuser \
  && adduser --uid 1000 --gid 1000 --disabled-password -gecos "" appuser

COPY entrypoint.sh /usr/local/bin/entrypoint.sh
RUN sed -i 's/\r$//' /usr/local/bin/entrypoint.sh && chmod 0755 /usr/local/bin/entrypoint.sh

COPY . .

RUN chown -R appuser:appuser /app

USER appuser

EXPOSE 5000

ENTRYPOINT ["entrypoint.sh"]

```

4.1 Multi-stage build – separacja build-time i runtime

Dockerfile wykorzystuje **multi-stage build**, czyli technikę polegającą na użyciu **wielu obrazów bazowych** w jednym procesie budowania.

```

[builder stage] → [runtime stage]

(cieężki)         (lekki)

```

Dlaczego to jest dobry wzorzec?

1. zależności (pip, gcc, nagłówki) są potrzebne **tylko podczas budowania**
2. finalny obraz: jest **mniejszy**, ma **mniejszą powierzchnię ataku**, szybciej się pobiera i uruchamia (ważne w Kubernetesie)

4.2 WORKDIR – kontekst pracy w obrazie

```
WORKDIR /app
```

1. ustawia katalog roboczy **dla wszystkich kolejnych instrukcji**
2. jeśli katalog nie istnieje → Docker go utworzy
3. eliminuje konieczność używania ścieżek absolutnych (/app/...)

4.3 Virtualenv w obrazie – izolacja zależności Pythona

```
RUN python -m venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"
```

1. Tworzony jest **virtualenv** w /opt/venv
2. Zmienna PATH zostaje zmodyfikowana: python, pip wskazują teraz na wersje z virtualenv

Dlaczego to dobre podejście?

1. brak konfliktów z systemowym Pythonem
2. pełna kontrola nad zależnościami
3. łatwe przeniesienie całego środowiska (COPY --from=builder)

To podejście jest znacznie lepsze niż instalowanie pakietów globalnie przez `pip install`.

4.4 COPY requirements.txt i cache warstw Dockera

```
COPY requirements.txt .
```

To kluczowa decyzja optymalizacyjna.

Docker buduje obrazy warstwowo, jeśli requirements.txt się **nie zmieni** Docker użyje **cache** i **nie uruchomi ponownie pip install**

4.5 Instalacja zależności – deterministyczny build

```
RUN pip install --upgrade pip \
&& pip install --no-cache-dir -r requirements.txt --timeout=100 --
index-url=https://pypi.python.org/simple
```

1. --no-cache-dir → brak cache pip → mniejszy obraz
2. jawny --index-url → deterministyczne źródło pakietów
3. timeout → stabilniejsze buildy w CI

To jest **sprawdzony** sposób instalowania zależności.

4.6 Drugi etap – minimalny runtime image

```
FROM python:3.13-slim
```

To **nowy etap** – nie dziedziczy nic z poprzedniego.

4.7 Czyszczenie apt – higiena warstw

```
RUN apt-get update \
&& rm -rf /var/lib/apt/lists/*
```

1. apt-get update jest wymagane przed instalacją pakietów
2. usunięcie /var/lib/apt/lists zmniejsza rozmiar obrazu

4.8 Przenoszenie virtualenv między etapami

```
COPY --from=builder /opt/venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"
```

To **główna idea multi-stage builda**:

1. kopiujemy **gotowe środowisko Pythona**
2. nie kopiujemy narzędzi buildowych
3. runtime image zawiera tylko to, co potrzebne

4.9 Uruchamianie jako non-root (bezpieczeństwo)

```
RUN addgroup -gid 1000 appuser \
&& adduser --uid 1000 --gid 1000 --disabled-password -gecos "" appuser

RUN chown -R appuser:appuser /app

USER appuser
```

Dlaczego to ważne?

1. kontener uruchomiony jako root: może eskalować uprawnienia, jest ryzykiem w Kubernetesie
2. standard bezpieczeństwa (CIS, Kubernetes benchmarks)

W K8s:

1. PodSecurityStandards **wymagają non-root**
2. ten Dockerfile jest z nimi zgodny

4.10 ENTRYPOINT – kontrola cyklu życia kontenera

```
COPY entrypoint.sh /usr/local/bin/entrypoint.sh
RUN sed -i 's/\r$//' /usr/local/bin/entrypoint.sh && chmod 0755
/usr/local/bin/entrypoint.sh

ENTRYPOINT ["entrypoint.sh"]
```

ENTRYPOINT:

1. **zawsze** się wykonuje
2. jest nadrzędny wobec CMD

5. Wprowadzenie teoretyczne – Kubernetes / Helm

Kubernetes to system orkiestracji kontenerów, szczegółowy opis k8s (nazwy można stosować wymiennie) dostępny jest przez oficjalną dokumentację, zalecane jest w razie niepewności teoretycznych korzystanie z poniższego linku w celu ich wyjaśnienia:

<https://kubernetes.io/docs/concepts/>

5.1 Czym jest Helm i jaki problem rozwiązuje

Helm jest **menedżerem pakietów dla Kubernetes**, często porównywanym do:

1. apt / yum w systemach Linux,
2. npm w ekosystemie JavaScript,
3. pip w Pythonie.

Jego głównym celem jest **uproszczenie zarządzania aplikacjami działającymi w Kubernetes**, poprzez:

1. pakowanie manifestów YAML w logiczne jednostki (charts),
2. parametryzację konfiguracji,
3. wersjonowanie wdrożeń,
4. ułatwienie aktualizacji (upgrade) i usuwania (uninstall).

Bez Helm, wdrażanie aplikacji w Kubernetes wymaga ręcznego:

1. utrzymywania wielu plików YAML,
2. powielania konfiguracji między środowiskami,
3. pilnowania kolejności zasobów,
4. ręcznego rollbacku w przypadku błędów.

Helm rozwiązuje te problemy, wprowadzając **warstwę abstrakcji nad surowymi manifestami Kubernetes**.

<https://helm.sh/docs/>

5.2 Helm jako warstwa deklaratywna nad Kubernetes

Kubernetes sam w sobie jest systemem **deklaratywnym** – opisujemy *jaki ma być stan*, a nie *jak do niego dojść*. Helm idzie o krok dalej:

- Kubernetes rozumie **pojedyncze zasoby** (Deployment, Service, ConfigMap),
- Helm rozumie **aplikację jako całość**.

Dzięki temu:

1. jedna aplikacja = jeden release Helm,
2. aktualizacja aplikacji = helm upgrade,
3. usunięcie aplikacji = helm uninstall.

Helm **nie zastępuje Kubernetes**, lecz:

1. generuje manifesty YAML,
2. przekazuje je do API Kubernetes,
3. śledzi, które zasoby należą do danego release'u.

5.3 Release – kluczowe pojęcie w Helm

Każde wdrożenie Helm tworzy **release**, czyli:

1. instancję charta,
2. z konkretnymi wartościami (values.yaml),
3. w określonym namespace,
4. pod unikalną nazwą.

Przykład:

```
helm install kube-lab ./devops/kube-lab -n kube-lab
```

W tym przypadku:

1. chart: kube-lab
2. release name: kube-lab
3. namespace: kube-lab

Helm zapisuje metadane release'u w klastrze, co umożliwia:

1. sprawdzenie aktualnego stanu (helm status),
2. podejrzenie zrenderowanych manifestów (helm get manifest),
3. rollback do poprzedniej wersji (helm rollback).

5.4 Helm a środowiska (dev / test / prod)

Jedną z największych zalet Helm jest **łatwa obsługa wielu środowisk**.

Ten sam chart:

1. **nie zmienia się** między środowiskami,
2. zmieniają się tylko wartości (values).

Przykład:

1. values.yaml – wartości domyślne,
2. values-dev.yaml – środowisko developerskie,
3. values-prod.yaml – środowisko produkcyjne.

Helm łączy wartości według kolejności:

```
helm install kube-lab ./chart -f values.yaml -f values-dev.yaml
```

5.5 Helm jako narzędzie CI/CD

Helm jest powszechnie wykorzystywany w pipeline'ach CI/CD, ponieważ:

1. jest deterministyczny,
2. działa idempotentnie,
3. integruje się z Kubernetes API.

Typowy workflow:

1. build obrazu Dockera,
2. push do registry,
3. helm upgrade --install,
4. monitoring rollout'u.

W tym laboratorium Helm pełni podobną rolę, przez jeden centralny punkt dostępu możliwe jest uruchomienie lokalne całego systemu, w CI/CD idea jest bardzo podobna tylko że środowiskiem docelowym zwykle jest jakaś chmura.

5.6 Helm Charts – struktura i koncepcja

Czym jest Helm Chart?

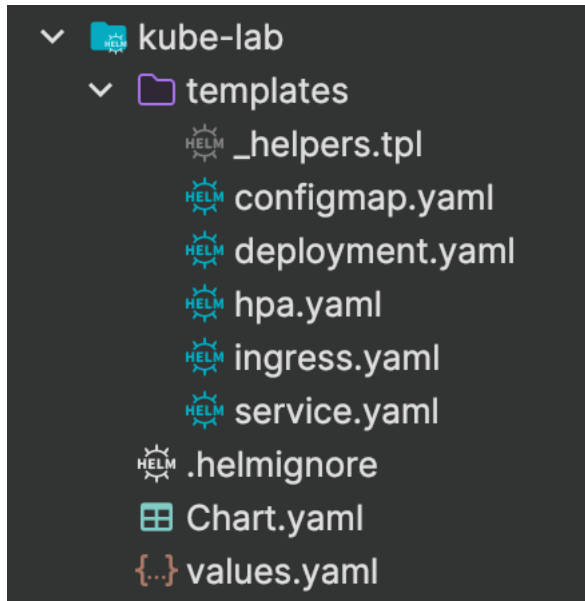
Helm Chart to:

1. **pakiet aplikacji Kubernetes,**
2. zawierający: szablony manifestów YAML, wartości konfiguracyjne, metadane.

Chart jest **niezależny od środowiska**, a jego zachowanie kontrolują wartości (values).

Podstawowa struktura katalogów charta

Typowa struktura charta wygląda następująco:



Znaczenie katalogów:

1. templates/ – szablony zasobów Kubernetes,
2. values.yaml – domyślna konfiguracja,
3. Chart.yaml – metadane charta,
4. _helpers.tpl – funkcje i fragmenty wspólne.

5.6.1 Chart.yaml – metadane charta

Plik Chart.yaml opisuje **czym jest chart**, a nie *jak działa aplikacja*.

```
apiVersion: v2
name: kube-lab
description: A Helm chart for Kubernetes
type: application
version: 0.1.0
appVersion: "latest"
```

Najważniejsze pola:

1. apiVersion – wersja API Helm,
2. name – nazwa charta,
3. version – wersja charta (logika wdrożeniowa),
4. appVersion – wersja aplikacji.

Rozdzielenie:

1. version → zmiany w Helm (np. nowe zasoby),
2. appVersion → wersja obrazu/aplikacji.

5.6.2 values.yaml – źródło konfiguracji

values.yaml jest **centralnym miejscem konfiguracji charta**.

```
fullnameOverride: kube-lab

api:
  name: "kube-lab"
  component: "api"
  replicas: 2
  image:
    repository: your-repository
    tag: ""
    pullPolicy: IfNotPresent
  podAnnotations: { }
  resources: { }
  strategy: { }
  containerPort: 5000
  livenessProbe:
    httpGet:
      path: /livez
      port: 5000
    initialDelaySeconds: 20
    periodSeconds: 10
  readinessProbe:
    httpGet:
      path: /readyz
      port: 5000
    initialDelaySeconds: 5
    periodSeconds: 5
  nodeSelector: { }
  tolerations: [ ]
  affinity: { }
  service:
    name: "kube-lab"
    component: "api-svc"
    type: ClusterIP
    port: 80
    annotations: { }
    labels: { }
  ingress:
    enabled: true
    name: "kube-lab"
    component: "api-ingress"
    className: "nginx"
    annotations: { }
    hosts:
      - kube-lab-api.127.0.0.1.nip.io
    path: /
    tls:
      enabled: True
      secretName: tls-secret
  secret:
    name: api-secret

config:
  name: "kube-lab"
  component: "config"
  APP_NAME: "FlaskLabApp"
  APP_ENV: "dev"
  APP_GREETING: "Hello from Flask running in Kubernetes via Ingress!"

autoscaling:
  enabled: False
  name: "kube-lab"
  component: "api-hpa"
  minReplicas: 4
```

```
maxReplicas: 10
targetCPUUtilizationPercentage: 80
targetMemoryUtilizationPercentage: 80
```

Zawiera:

1. liczbę replik,
2. parametry obrazu,
3. konfigurację zasobów,
4. ustawienia ingress, autoscaling itd.

Cechy:

1. wartości są **czysto deklaratywne**,
2. nie zawierają logiki,
3. mogą być nadpisywane przez inne pliki lub CLI.

Helm **nie waliduje semantycznie** values – ich poprawność zależy od szablonów.

5.6.3 _helpers.tpl – DRY w Helm

Plik _helpers.tpl służy do:

1. definiowania funkcji pomocniczych,
2. standaryzacji nazw,
3. generowania etykiet (labels).

```
{{- define "kube-lab.name" -}}
{{ default .Chart.Name .Values.nameOverride | trunc 63 | trimSuffix "-" }}
{{- end }}

{{ define "kube-lab.fullname" -}}
{{ $name := default .Chart.Name .Values.nameOverride -}}
{{ printf "%s-%s" $name .Release.Name | trunc 63 | trimSuffix "-" }}
{{- end }}

{{ define "kube-lab.labels" -}}
app.kubernetes.io/name: {{ include "kube-lab.name" . }}
app.kubernetes.io/instance: {{ .Release.Name }}
app.kubernetes.io/version: {{ .Chart.AppVersion }}
app.kubernetes.io/managed-by: {{ .Release.Service }}
{{- end }}
```

5.7 Templates – dynamiczne generowanie YAML

Katalog templates/ w Helm chart zawiera **szablony manifestów Kubernetes**, które są renderowane przez Helm na podstawie wartości z plików values.yaml.

Każdy plik w tym katalogu odpowiada zazwyczaj **jednemu typowi zasobu Kubernetes** (np. Service, Deployment, Ingress).

Helm:

1. podstawia wartości z `.Values`,
2. renderuje finalne manifesty YAML,
3. wysyła je do API Kubernetes.

W tej sekcji omówiono kluczowe template'y używane w laboratorium.

5.7.1 Service

Rola Service w Kubernetes

Service zapewnia **stabilny punkt dostępu do Podów**, niezależny od:

1. ich liczby,
2. ich nazw,
3. restartów.

Każdy Service:

1. otrzymuje **stały adres IP w klastrze**,
2. jest dostępny pod nazwą DNS:
`service-name.namespace.svc.cluster.local`.

Dla aplikacji HTTP w Kubernetes:

1. Service typu ClusterIP obsługuje ruch **wewnątrz klastra**,
2. ruch z zewnątrz obsługuje **Ingress**, który kieruje go do Service.

Template Service w laboratorium

```
apiVersion: v1
kind: Service
metadata:
  name: {{ include "kube-lab.fullname" . }}
```

Helm generuje nazwę Service na podstawie:

1. nazwy charta,
 2. nazwy release'u,
- co zapewnia **unikalność zasobów w klastrze**.

Labels i annotations

```
labels:
  app.kubernetes.io/name: {{ .Values.api.service.name | quote }}
  app.kubernetes.io/component: {{ .Values.api.service.component | quote }}
```

Labels:

1. służą do identyfikacji zasobów,
2. są używane m.in. przez selektory oraz narzędzia monitorujące.

Annotations:

1. umożliwiają przekazanie metadanych (np. Prometheus scraping),
2. są opcjonalne i kontrolowane przez values.yaml.

Selector i porty

```
selector:
  app.kubernetes.io/name: {{ .Values.api.name | quote }}
  app.kubernetes.io/instance: {{ .Release.Name | quote }}
```

Selector:

1. łączy Service z Podami,
2. musi dokładnie odpowiadać labelom Podów z Deploymentu.

```
ports:
- name: http
  port: {{ .Values.api.service.port }}
  targetPort: http
```

1. port → port wystawiony przez Service,
2. targetPort → port kontenera w Podzie.

5.7.2 Deployment

Kontroler utrzymujący żadaną liczbę replik i umożliwiający rolling update. To Deployment „posiada” ReplicaSety, a ReplicaSety tworzą Pody.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ include "kube-lab.fullname" . }}
  labels:
    app: {{ default .Values.api.name | quote }}
    name: {{ default .Values.api.name | quote }}
    app.kubernetes.io/name: {{ default .Values.api.name | quote }}
    {{ include "kube-lab.labels" . | nindent 4 }}
spec:
  replicas: {{ .Values.api.replicas }}
  {{- with .Values.api.strategy }}
  strategy:
    {{ toYaml . | nindent 4 }}
  {{- end }}
  selector:
    matchLabels:
      app.kubernetes.io/name: {{ default .Values.api.name .Values.api.name | quote }}
      app.kubernetes.io/instance: {{ .Release.Name | quote }}
  template:
    metadata:
      labels:
        app.kubernetes.io/name: {{ default .Values.api.name .Values.api.name | quote }}
        app.kubernetes.io/instance: {{ .Release.Name | quote }}
      {{- with .Values.api.podAnnotations }}
      annotations:

```

```

{{ toYaml . | nindent 8 }}
{{- end }}
spec:
  containers:
    - name: {{ default .Values.api.name .Values.api.name }}
      image: "{{ .Values.api.image.repository }}:{{ .Values.api.image.tag }}"
      imagePullPolicy: {{ .Values.api.image.pullPolicy }}
      ports:
        - name: http
          containerPort: {{ .Values.api.containerPort }}
      envFrom:
        - configMapRef:
            name: {{ include "kube-lab.fullname" . }}
        - secretRef:
            name: {{ .Values.api.secret.name }}
      livenessProbe:
        {{ toYaml .Values.api.livenessProbe | nindent 12 }}
      readinessProbe:
        {{ toYaml .Values.api.readinessProbe | nindent 12 }}
      {{- with .Values.api.resources }}
      resources:
        {{ toYaml . | nindent 12 }}
      {{- end }}
      {{- with .Values.api.nodeSelector }}
      nodeSelector: {{ toYaml . | nindent 8 }}
      {{- end }}
      {{- with .Values.api.affinity }}
      affinity: {{ toYaml . | nindent 8 }}
      {{- end }}
      {{- with .Values.api.tolerations }}
      tolerations: {{ toYaml . | nindent 8 }}
      {{- end }}

```

5.7.3 Ingress

Warstwa routingu HTTP(S) do Service, obsługiwana przez **Ingress Controller** (np. NGINX Ingress). W labie aplikacja jest osiągalna przez host:

<https://kube-lab-api.127.0.0.1.nip.io>

```

{{- if .Values.api.ingress.enabled }}
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: {{ include "kube-lab.fullname" . }}
  labels:
    app.kubernetes.io/name: {{ .Values.api.ingress.name | quote }}
    app.kubernetes.io/component: {{ .Values.api.ingress.component | quote }}
    {{ include "kube-lab.labels" . | nindent 4 }}
  {{- with .Values.api.ingress.annotations }}
  annotations:
    {{ toYaml . | nindent 4 }}
  {{- end }}
spec:
  {{- if .Values.api.ingress.className }}
  ingressClassName: {{ .Values.api.ingress.className | quote }}
  {{- end }}
  {{- if .Values.api.ingress.tls.enabled }}
  tls:
    - hosts:
        {{- range .Values.api.ingress.hosts }}
        - {{ . | quote }}
        {{- end }}
      secretName: {{ .Values.api.ingress.tls.secretName | quote }}
  {{- end }}
  rules:
    {{- range .Values.api.ingress.hosts }}
    - host: {{ . | quote }}
      http:
        paths:
          - path: {{ $.Values.api.ingress.path | default "/" | quote }}
            pathType: Prefix
            backend:

```



```

      service:
        name: {{ include "kube-lab.fullname" $ }}
        port:
          number: {{ $.Values.api.service.port }}
    {{- end }}
  {{- end }}

```

5.7.4 ConfigMap

Konfiguracja niewrażliwa (np. APP_GREETING). W labie Helm renderuje ConfigMap z values.yaml, a K8s wstrzykuje wartości jako env vars.

```

apiVersion: v1
kind: ConfigMap
metadata:
  name: {{ include "kube-lab.fullname" . }}
  labels:
    app: {{ default .Values.config.name | quote }}
    name: {{ default .Values.config.name | quote }}
    app.kubernetes.io/name: {{ default .Values.config.name | quote }}
    {{ include "kube-lab.labels" . | nindent 4 }}
data:
  APP_NAME: {{ .Values.config.APP_NAME | quote }}
  APP_ENV: {{ .Values.config.APP_ENV | quote }}
  APP_GREETING: {{ .Values.config.APP_GREETING | quote }}

```

Koniec Wprowadzenia